Namal University Mianwali

Department of Computer Science

Course	Advanced Database Systems (CSC-371)
Assignment No.	02
Assignment Title	System Analysis, Schema Design & Implementation Strategy
Student 1	Anila Younas (NUM-BSCS-2023-05)
Student 2	Shehr Bano (NUM-BSCS-2023-11)
Submitted To	Dr. Muzamil Ahmed
Date	06/04/2026

Table of Contents

1. Refined Project Title

QuickBite: A Polyglot Database Architecture for Distributed Real-Time Food Delivery and Order Management

2. Similar Existing Systems

There is an analysis of four existing food-delivery platforms in terms of databases. The database perspective is considered instead of the application features in comparison [1].

2.1 Uber Eats

- **Problem Solved:** Matches consumers, restaurants and couriers with on-demand food delivery with dynamic ETA and surge pricing in world markets [1].
- **Users:** Customers who make orders, restaurant partners who fulfil orders, delivery drivers, managers of fleet operations and finance teams who reconcile payments.
- **Major DB Operations:** 100 Hz real-time GPS writes per driver in operation; full ACID order state transitions; finalisation of payment; demand-surge analytics.
- **Data Handled:** Transactional order data, high-velocity geolocation data, user accounts and credentials, restaurant menus, payment and audit data.
- **Scale:** Hundreds of millions of orders annually, in 45+ countries. The GPS ingestion creates billions of write operations per day [2].
- **Possible DB Technologies:** PostgreSQL or MySQL to provide core OLTP, Apache Cassandra to provide location ingestion, Redis to provide session caching and pub/sub dispatch, Kafka to provide event streaming, Elasticsearch to provide menu search [2].

2.2 Foodpanda

- **Problem Solved:** Multi-country food ordering and vendor onboarding, multi-currency payment, and localised menus in 40+ markets in Asia and Europe [1].
- **Users:** consumers, local restaurant vendors having heterogeneous menu structures, delivery riders, city-level operation teams, and payment settlement teams.
- **Major DB Operations:** schema-variable vendor catalog ingestion; multi-currency ACID transactions; rider assignment and GPS tracking; order status management.
- **Data Handled:** Semi-structured vendor catalogs that include restaurant-specific fields, financial settlements, geolocation streams, mobile payment events.
- **Scale:** 40+ countries with timezone-staggered peak loads. The size of vendor catalogue differs tremendously among markets [3].
- **Possible DB Technologies:** MySQL sharded cluster to run OLTP, MongoDB to have flexible vendor catalogues, Redis pub/sub to rider dispatch, Snowflake or BigQuery to report on analytics [3].

2.3 DoorDash

- **Problem Solved:** Hyperlocal food delivery in North America using machine-learning ETA prediction, dynamic dasher routing, and dispute resolution workflows [4].
- **Users:** Consumers, merchant partners, dashers (delivery drivers), operations analysts, and compliance teams to resolve payment disputes.
- **Major DB Operations:** ML-based recalculation of ETA; querying surge pricing on live order density; ACID payment finality; geospatial nearest-dasher search; querying dispute resolution records [4].
- **Data Handled:** Order lifecycle events, real-time GPS telemetry, merchant inventory counters, customer feedback and time-series performance metrics of delivery.
- **Scale:** Tens of millions of monthly orders. It has been reported to handle millions of database transactions per day in a geo-distributed infrastructure [4].

- **Possible DB Technologies:** PostgreSQL to support ACID orders, CockroachDB to support geo-distributed strong consistency, Apache Flink to support real-time stream processing, Elasticsearch to support full-text menu and merchant search [4].

2.4 Deliveroo

- **Problem Solved:** Restaurant-quality, high-end food delivery in Europe and Asia, with the ability to batch orders (many orders per rider) and track SLA compliance [1].
- **Users:** Consumers, restaurant kitchens that deal with SLA requirements, riders that deal with batched deliveries, city managers that deal with compliance, and cross-border finance departments.
- **Major DB Operations:** Batch assembly of orders; calculating dynamic prices; clearing of cross-border payments by converting currencies; SLA contract evaluation; rider performance measurements.
- **Data Handled:** Order batches, rider telemetry, restaurant SLA contracts, structured financial records, cross-border settlement data.
- **Scale:** Operation in 10+ countries. Order batching makes the write operation much more complex; one rider update can cause many changes in the state of order [3].
- **Possible DB Technologies:** PostgreSQL to handle ACID orders and payments and MongoDB to handle restaurant catalogs and rider telemetry and Redis to handle rate limiting and caching and BigQuery to handle analytics [3].

3. Comparative Analysis of Similar Systems

System	Main Features	Data Complexity	Scalability Need	Possible DB Technology
Uber Eats	Real-time GPS, surge pricing, ETA ML, global scale	High — OLTP + streaming + geospatial	Global, hundreds of millions/year	PostgreSQL, Cassandra, Redis, Kafka, Elasticsearch
Foodpanda	Multi-country catalogs, multi-currency payments	High — schema variation per vendor	Multi-region, timezone peaks	MySQL sharded, MongoDB, Redis, Snowflake
DoorDash	ML ETA, dasher routing, dispute resolution	Very High — ML + OLTP + geospatial	Tens of millions/month, geo-distributed	PostgreSQL, CockroachDB, Flink, Elasticsearch
Deliveroo	Order batching, SLA tracking, cross-border payments	High — batch + contract + financial	Multi-country, SLA-bound writes	PostgreSQL, MongoDB, Redis, BigQuery
QuickBite (This Project)	State-machine orders, polyglot sync, ETA aggregation	High — Oracle ACID + MongoDB flexible docs	Platform-scale demo + optional cloud tier	Oracle XE, MongoDB 7.x, Azure Cosmos DB

Key Observations

- **Common Features:**

All four systems separate ACID transactional order data and high-velocity eventual-consistent telemetry. All platforms employ at least two distinct database paradigms, and thereby support the polyglot persistence paradigm that QuickBite adopts [5].

- **Common Database Challenges:**

The amplification of GPS writes, concurrent inventory loss, cross-system eventual consistency control, and deterioration of ETA in peak load are consistently observed across all platforms analysed [2][4].

- **Limitations in Existing Platforms:**

DoorDash and Deliveroo will have to have dedicated site-reliability teams to handle Cassandra and CockroachDB cluster tuning. None of the platforms expose academic-level transparency in their cross-database synchronisation mechanisms [4].

- **Ideas Adopted for QuickBite:**

The outbox pattern adopted by DoorDash and Deliveroo is based on transactional outbox, meaning a heavyweight Kafka message brokers are substituted with a lightweight scheduled synchronisation job, which is appropriate in a semester-scale project. GPS expiry MongoDB TTL indexes are based on patterns reported in Uber Eats-scale systems. The catalogue-based approach to compound indexes on restaurant catalogs resembles the design of Foodpanda vendor catalogues [3][8].

4. Detailed Database Schema Design

4.1 Oracle XE — Relational / ACID Layer

There are seven tables in the Oracle XE schema. ORDERS table is range-partitioned along ORDER_DATE (monthly partitions) to allow historical queries to be sped up by partition pruning. Row-level locking is used to place orders on the same restaurant in concurrency without contention at the table level [2][7].

Table	Attribute	Data Type	Keys / Constraints
USERS	user_id	NUMBER	PRIMARY KEY — auto-increment via sequence
	full_name	VARCHAR2 (100)	NOT NULL
	email	VARCHAR2 (150)	UNIQUE, NOT NULL
	password_hash	VARCHAR2 (255)	NOT NULL — bcrypt hash stored
	role	VARCHAR2 (20)	CHECK (role IN ('CUSTOMER','RIDER','RESTAURANT','ADMIN'))
	created_at	TIMESTAMP	DEFAULT SYSDATE
RESTAURANTS	rest_id	NUMBER	PRIMARY KEY
	owner_id	NUMBER	FK → USERS(user_id), NOT NULL
	name	VARCHAR2 (150)	NOT NULL
	city_zone	VARCHAR2 (50)	NOT NULL — used as MongoDB shard key anchor
	latitude	NUMBER(10,7)	NOT NULL
	longitude	NUMBER(10,7)	NOT NULL
	is_active	NUMBER(1)	DEFAULT 1, CHECK (is_active IN (0,1))
ORDERS	order_id	NUMBER	PRIMARY KEY
	cust_id	NUMBER	FK → USERS(user_id), NOT NULL

Table	Attribute	Data Type	Keys / Constraints
	rest_id	NUMBER	FK → RESTAURANTS(rest_id), NOT NULL
	rider_id	NUMBER	FK → USERS(user_id), NULLABLE until assigned
	status	VARCHAR2 (20)	CHECK (status IN ('PLACED','CONFIRMED','PREPARING','PICKED_UP','DELIVERED','CANCELLED'))
	order_date	DATE	NOT NULL — Range Partition Key (monthly)
	total_amount	NUMBER(10,2)	NOT NULL, CHECK (total_amount > 0)
ORDER_ITEMS	item_id	NUMBER	PRIMARY KEY
	order_id	NUMBER	FK → ORDERS(order_id), NOT NULL
	menu_item_name	VARCHAR2 (200)	NOT NULL
	quantity	NUMBER	NOT NULL, CHECK (quantity > 0)
	unit_price	NUMBER(8,2)	NOT NULL
PAYMENTS	pay_id	NUMBER	PRIMARY KEY
	order_id	NUMBER	FK → ORDERS(order_id), UNIQUE — one payment per order
	amount	NUMBER(10,2)	NOT NULL
	method	VARCHAR2 (30)	CHECK (method IN ('CARD','WALLET','CASH','UPI'))
	status	VARCHAR2 (20)	CHECK (status IN ('PENDING','COMPLETED','REFUNDED'))
	paid_at	TIMESTAMP	DEFAULT SYSDATE
OUTBOX_EVENTS	event_id	NUMBER	PRIMARY KEY
	order_id	NUMBER	FK → ORDERS(order_id)
	event_type	VARCHAR2 (50)	e.g. ORDER_STATUS_CHANGED, ORDER_PLACED
	payload	CLOB	JSON blob of changed fields
	is_dispatched	NUMBER(1)	DEFAULT 0 — set to 1 after MongoDB sync
	created_at	TIMESTAMP	DEFAULT SYSDATE
AUDIT_LOG	log_id	NUMBER	PRIMARY KEY
	order_id	NUMBER	FK → ORDERS(order_id)

Table	Attribute	Data Type	Keys / Constraints
	changed_by	NUMBER	FK → USERS(user_id)
	old_status	VARCHAR2 (20)	
	new_status	VARCHAR2 (20)	
	changed_at	TIMESTAMP	DEFAULT SYSDATE

4.2 MongoDB 7.x — Document / Flexible Layer

The document layer is made up of four collections. Embedding is also favored to referencing in the restaurants collection since the enormous percentage of reads will retrieve the whole restaurant catalog with one operation avoiding application-side joins [6].

Collection	Key Fields	Design Decisions
restaurants	restaurant_id, name, city_zone, cuisine[ ], menu[ ] (embedded), location (GeoJSON), opening_hours, avg_rating, is_active	Full menu EMBEDDED inside one document — single read fetches entire catalog without joins. Compound index on (city_zone, cuisine, is_active). Sharded by city_zone [6].
riderlocations	rider_id, location (GeoJSON Point), timestamp, status (AVAILABLE|ASSIGNED), city_zone	2dsphere index enables geospatial $near dispatch queries. TTL index expires records older than 48 hours automatically. Sharded by city_zone to distribute GPS write load [6].
orderhistory	oracle_order_id, cust_id, rest_id, status, items[ ] (embedded), total_amount, eta_minutes, updated_at	Populated via outbox sync from Oracle. Unique index on oracle_order_id ensures idempotent upserts. Write concern: majority prevents data loss on primary failover [6][8].
eta_estimates	restaurant_id, hour_of_day, avg_prep_minutes, sample_count, last_updated	Materialized result of aggregation pipeline; refreshed every 30 minutes. Compound index on (restaurant_id, hour_of_day) gives O(1) ETA lookup at order placement time [6].

4.3 Schema Diagram

The entire schema of both database layers, comprising all the entity relationships, primary and foreign keys, and the outbox synchronisation connection between the Oracle and MongoDB, is provided in figure 1.

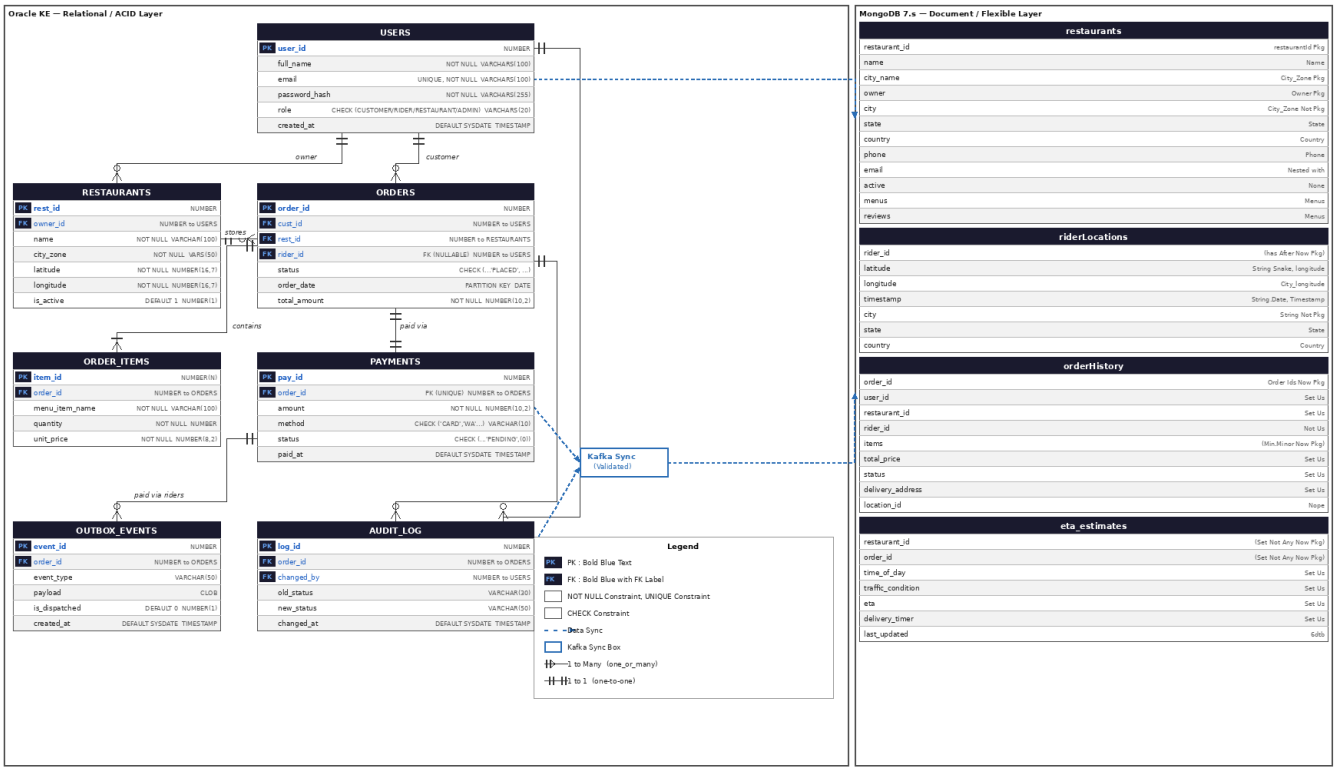

Figure 1: QuickBite Database Schema

5. Core Transactions and Queries

There are six transactions which represent all the key data flows of the system. The choice gives preference to those operations that are specific to this architecture or those that exhibit concurrency and consistency issues. All the queries are targeted to Oracle XE and MongoDB 7.x described in the schema [2][6][7].

T1 — Place New Order (Oracle ACID Transaction)

Purpose: Add a new order with its line items, a pending payment record and an outbox event in one Oracle operation. Any failure backrolls up all four inserts.

T1 — Place New Order (Oracle ACID Transaction)

```
BEGIN
  INSERT INTO ORDERS (cust_id, rest_id, status, order_date, total_amount)
  VALUES (:p_cust_id, :p_rest_id, 'PLACED', SYSDATE, :p_total);

  INSERT INTO ORDER_ITEMS (order_id, menu_item_name, quantity, unit_price)
  VALUES (order_seq.CURRVAL, :p_item_name, :p_qty, :p_unit_price);

  INSERT INTO PAYMENTS (order_id, amount, method, status)
  VALUES (order_seq.CURRVAL, :p_total, :p_method, 'PENDING');

  INSERT INTO OUTBOX_EVENTS (order_id, event_type, payload, is_dispatched)
  VALUES (order_seq.CURRVAL, 'ORDER_PLACED', :p_json_payload, 0);

  COMMIT;
EXCEPTION
  WHEN OTHERS THEN ROLLBACK; RAISE;
```

```
END;
```

Note: It is the most critical transaction in the system. The assurances in ACID make it impossible to charge a customer without an order record, and impossible to have an order without an outbox event, which will ultimately be written to MongoDB [7][8].

T2 — Order Status State Machine (PL/SQL Stored Procedure)

Purpose: Enforce valid state transitions through a PL/SQL procedure. Invalid transitions (e.g., PLACED → DELIVERED) are explicitly rejected with an application error. An AFTER UPDATE trigger on ORDERS automatically records to OUTBOX_EVENTS and AUDIT_LOG.

T2 — Order Status State Machine (PL/SQL Stored Procedure)

```
CREATE OR REPLACE PROCEDURE ORDER_STATE_MACHINE(
  p_order_id IN NUMBER,
  p_new_status IN VARCHAR2
) AS
  v_cur_status VARCHAR2(20);
BEGIN
  SELECT status INTO v_cur_status
  FROM ORDERS WHERE order_id = p_order_id FOR UPDATE;

  IF  (v_cur_status = 'PLACED'     AND p_new_status = 'CONFIRMED')
   OR (v_cur_status = 'CONFIRMED'  AND p_new_status = 'PREPARING')
   OR (v_cur_status = 'PREPARING'  AND p_new_status = 'PICKED_UP')
   OR (v_cur_status = 'PICKED_UP'  AND p_new_status = 'DELIVERED')
   OR (p_new_status = 'CANCELLED'
       AND v_cur_status NOT IN ('DELIVERED','CANCELLED'))
  THEN
    UPDATE ORDERS SET status = p_new_status
    WHERE order_id = p_order_id;
    COMMIT;
  ELSE
    RAISE_APPLICATION_ERROR(-20001,
      'Invalid transition: ' || v_cur_status || ' -> ' || p_new_status);
  END IF;
END;
```

Note: The FOR_UPDATE clause acquires a row level lock and then reads the status of the row itself to guard against the possibility of two parallel sessions mutating the same order at the same time - a critical race condition that is found to be a weakness in application-code state management in such systems [2][7].

T3 — Rider GPS Update (MongoDB — High-Frequency Write)

Purpose: Add a new GPS position record of an active rider. This is the most frequency a write in the whole system, and it happens every 10 seconds per active rider. It should not interfere with the Oracle layer.

T3 — Rider GPS Update (MongoDB — High-Frequency Write)

```
db.riderlocations.insertOne({
  rider_id: 42,
  location: {
    type: "Point",
    coordinates: [71.8234, 32.5967]   // [longitude, latitude]
  },
  timestamp: new Date(),
```

```
  status: "AVAILABLE",
  city_zone: "Mianwali-Central"
});

// TTL index ensures documents older than 48 hours are auto-deleted:
// db.riderlocations.createIndex({ "timestamp": 1 }, { expireAfterSeconds: 172800 })
```

Note: The write amplification to the Oracle primary does not occur when storing GPS in MongoDB instead of Oracle. The TTL index imposes automatic expiry thus the collection does not increase ad infinitum over time [5][6].

T4 — Find Nearest Available Rider (MongoDB Geospatial Query)

Purpose: During dispatching, identify the top 3 nearest AVAILABLE riders that are within a 5 km radius of the GPS position of the restaurant. Index on the collection of riderlocations.

T4 — Find Nearest Available Rider (MongoDB Geospatial Query)

```
db.riderlocations.find({
  status: "AVAILABLE",
  city_zone: "Mianwali-Central",
  location: {
    $near: {
      $geometry: {
        type: "Point",
        coordinates: [71.8, 32.6]   // restaurant coordinates
      },
      $maxDistance: 5000            // metres
    }
  }
}).limit(3);
```

Note: In the absence of the 2dsphere index this query reduces to scan over the collection at O(n), not O(log n). The city_zone filter is used to first limit the working set prior to the geospatial calculation to substantially cut the number of distance calculations needed [6].

T5 — ETA Aggregation Pipeline (MongoDB)

Purpose: Calculate the mean time to prepare meals each restaurant on average per hour of day using previous order data. The results are combined into the etaestimates collection to be looked up at O(1) at time of order placement.

T5 — ETA Aggregation Pipeline (MongoDB)

```
db.orderhistory.aggregate([
  { $match: { status: "DELIVERED" } },
  {
    $group: {
      _id: {
        rest_id:     "$rest_id",
        hour_of_day: { $hour: "$order_date" }
      },
      avg_prep_minutes: { $avg: "$eta_minutes" },
      sample_count:     { $sum: 1 }
    }
  },
  {
    $merge: {
      into: "eta_estimates",
```

```
    on:   ["_id.rest_id", "_id.hour_of_day"],
    whenMatched:  "replace",
    whenNotMatched: "insert"
   }
  }
]);
```

Note: This is in place of the fixed preparation-time estimates which have been determined as a limitation in existing platforms (Assignment 1, Section 2). It validates the pipeline using explain() to ensure that it is using compound indexes on (rest_id, hour of day) [6].

T6 — Outbox Synchronisation (Cross-Database Consistency)

Purpose: A scheduled job processes unprocessed outbox events in Oracle and inserts them into MongoDB orderhistory and marks each event sent. Idempotent: Running twice after a partial failure leaves no duplicates.

T6 — Outbox Synchronisation (Cross-Database Consistency)

```
-- Step 1: Oracle — fetch pending events (application layer reads this)
SELECT event_id, order_id, event_type, payload
FROM   OUTBOX_EVENTS
WHERE  is_dispatched = 0
ORDER  BY created_at
FETCH  FIRST 100 ROWS ONLY;

-- Step 2: For each event, upsert into MongoDB (application layer)
db.orderhistory.updateOne(
  { oracle_order_id: <event.order_id> },
  { $set: <JSON.parse(event.payload)> },
  { upsert: true }
);

-- Step 3: Oracle — mark event as dispatched
UPDATE OUTBOX_EVENTS
SET    is_dispatched = 1
WHERE  event_id = :p_event_id;
COMMIT;
```

Note: The MongoDB upsert ensures idempotency since a re-delivered event with the same oracle_order_id modifies, but does not create a duplicate record. FETCH FIRST 100 ROWS clause limits the amount of jobs to be run so that it does not accumulate lag in the face of high order volume [8].

6. Indexing and Performance Planning

DB	Table / Collection	Indexed Field(s)	Index Type	Justification
Oracle	ORDERS	order_date	Range Partition	Partition pruning eliminates full table scan for date-range queries. Historical reporting skips all partitions outside the requested range [7].
Oracle	ORDERS	cust_id, status	Composite B-Tree	Customer order history lookup — most frequent read pattern in the system.

DB	Table / Collection	Indexed Field(s)	Index Type	Justification
				Status filter applied within customer partition [2].
Oracle	ORDERS	rest_id, status	Composite B-Tree	Restaurant pending orders dashboard query. Allows restaurant operators to view only PLACED and CONFIRMED orders efficiently [7].
Oracle	OUTBOX_EVENTS	is_dispatched, created_at	Composite B-Tree	Outbox sync job scans only undispatched events ordered by time. Without this index, each job execution performs a full table scan [8].
Oracle	PAYMENTS	order_id	Unique B-Tree	Enforces one-payment-per-order business rule at the database level. Also accelerates payment status lookup by order ID [7].
Oracle	AUDIT_LOG	order_id	B-Tree	Dispute resolution queries retrieve all status changes for a specific order. Sequential scan of audit log would be prohibitive at scale [2].
MongoDB	restaurants	city_zone, cuisine, is_active	Compound	Filtered restaurant browse query — city zone applied first (high cardinality reduction), then cuisine type, then active flag. Exact field order is required [6].
MongoDB	riderlocations	location	2dsphere	Required for $near geospatial queries at dispatch time. Without this index, nearest-rider search degrades from O(log n) to O(n) [6].
MongoDB	riderlocations	timestamp	TTL (48 h)	Automatic expiry of stale GPS records. Prevents unbounded collection growth without requiring application-level cleanup jobs [6].
MongoDB	orderhistory	oracle_order_id	Unique	Ensures idempotent upserts during outbox sync. Duplicate event delivery updates rather than inserts, preventing data duplication [8].

DB	Table / Collection	Indexed Field(s)	Index Type	Justification
MongoDB	eta_estimates	restaurant_id, hour_of_day	Compound	O(1) ETA lookup at order placement. The two fields together form the composite lookup key used by the aggregation $merge stage [6].

Performance Risk Areas

1. **GPS Write Amplification:**

System writes the most volume riderlocations collection (one insert per active rider every 10 seconds). The absence of the TTL index causes the collection to become unlimited. In the absence of city zone sharding, all writes go to one MongoDB server [5][6].

2. **ETA Pipeline Degradation:**

With orderhistory expanding into millions of documents, the aggregation pipeline needs to be verified with explain() to ensure the use of compound indexing. The materialized eta_estimates collection does not allow the pipeline to be run on all orders [6].

3. **Oracle Partition Hotspot:**

All contemporary orders fell upon the same monthly division. In case write-contention is detected during load testing, sub-partitioning of rest-id in the existing-month partition will be considered as a remediation [7].

4. **Outbox Queue Growth:**

The is dispatched = 0 queue expands, in case the 30 second sync interval is not able to match the volume of orders. Each scan is limited by the composite index on (is_dispatched, created_at), though queue depth ought to be tracked as an operational measure [8].

7. Updated System Architecture

The architecture is based on the four-tier Assignment 1 architecture with the addition of the Auth Service, Sync Job, and explicit cross-tier data-flow labels. The revised set of components is represented in the diagram (Figure 2) and adheres to the pattern of transactional outbox in Richardson (2018) [8].

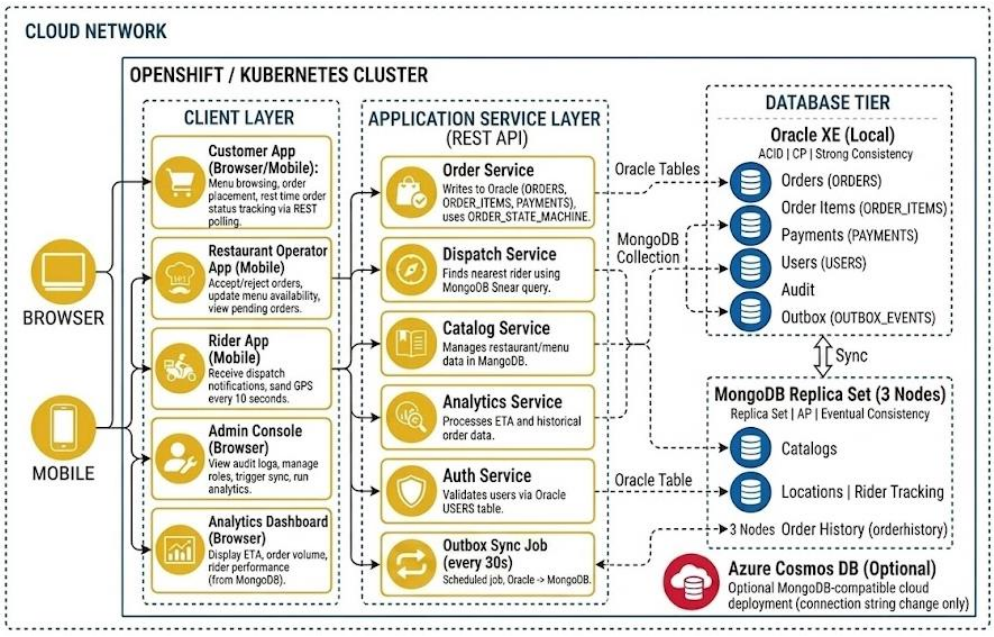

Figure 2: QuickBite Updated System Architecture Diagram

8. Security and Access Planning

Role	Accessible Data	Denied / Restricted
CUSTOMER	Own orders, own profile, restaurant catalogs, own payment status	Other customers' records, payment card details, audit log, admin functions
RIDER	Assigned orders only, own GPS history, restaurant pickup address	Customer personal data, payment details, other riders' assignments
RESTAURANT	Incoming orders for own rest_id, own menu documents in MongoDB	Customer personal data, payment records, other restaurants' data
ADMIN	All tables (read/write), AUDIT_LOG, OUTBOX_EVENTS, system configuration	Raw payment card numbers (masked at application layer before storage)
SYNC_JOB	OUTBOX_EVENTS (SELECT, UPDATE is_dispatched only)	All other Oracle tables, no DDL privileges, no DROP or TRUNCATE

Data Protection Measures

1. **Password Storage:**
   The user passwords have been stored in their bcrypt hash form (cost factor 12). In Oracle or MongoDB, plain-text passwords are not stored anywhere [1].
2. **Payment Data Masking:**
   The application layer masks card numbers; the last four digits and type of payment method are stored in PAYMENTS.
3. **Oracle Role-Based Access Control:**
   There are five database roles (app_customer, app_rider, app_restaurant, app_admin, sync_job). Application services interface with the least-privilege role to each operation [7].
4. **MongoDB Authentication:**
   The replica set is authenticated by username/password. The sync job runs under a special user of MongoDB that can only write to the orderhistory.
5. **Network Security:**
   Oracle XE and MongoDB do not expose themselves to outside the cluster network. All connections use TLS. There is no client application that links directly with the database layer.
6. **Audit Trail:**
   AUDIT_LOG provides a history of all changes in the status of orders with the name of the user who changed the status and allows the reconstruction of any questionable delivery or payment in a forensic way.

9. Implementation Plan

The plan is broken down into six big modules. Every module is associated with a functional area of the system. The modules are ordered in a manner that the subsequent modules can rely on the database tables and services developed by the preceding ones. User and Authentication Module is designed first due to all the other services, which require authenticated user identity and role check [1][7].

Phase	Weeks	Major Module	Tasks and Deliverables
1	1–2	User & Authentication Module	Create Oracle USERS table using role restrictions. Use password hashing (bcrypt) at the application layer. Build Auth Service (log in, token generation, verification of roles). Unit-test each of the four

Phase	Weeks	Major Module	Tasks and Deliverables
			roles. This module is created initially since all other modules will rely on authenticated user identity.
2	3	Restaurant & Menu Management Module	Create Oracle RESTAURANTS table. Create compound index and embedded menu document structure MongoDB restaurants collection. Build Catalog Service (CRUD menu, availability switching). Compare schema-variable menu structures in several types of restaurants.
3	4	Order Lifecycle Module	Create Oracle ORDERS, ORDER_ITEMS, PAYMENTS, OUTBOX_EVENTs, AUDIT log tables with all constraints, partitioning. Introduce ORDER_STATE_MACHINE PL/SQL procedure. Write AFTER UPDATE trigger for outbox and audit. Build Order Service. Test all valid and all invalid state transitions with unit tests.
4	5–6	Rider Dispatch & GPS Module	Create MongoDB riderlocations collection with 2dsphere and TTL indexes. Build Dispatch Service using $near query. Implement rider assignment workflow (Oracle rider_id update + MongoDB status update). Load test GPS write volume; check TTL expiry.
5	7-8	Cross-Database Synchronisation Module	Implement Outbox Sync Job. At-least-once delivery: Test with forced mid-batch failure. Check idempotents upsert - re-run job and ensure that there are no duplicates in MongoDB. Test outbox queue depth with simulated high order volume.
6	9	Analytics, ETA & Cloud Module	Run ETA aggregation pipeline; test with explain(); test query latency pre-indexing and post-indexing. CAP theorem proving (isolate one MongoDB secondary; see stale reads vs Oracle consistency). Provided that there is time, migrate MongoDB layer to Azure Cosmos DB using Student subscription.

Database Creation Sequence

1. Create Oracle users and five application roles with minimum-privilege grants.
2. Create DDL: USERS → RESTAURANTS → ORDERS (monthly range partition) → ORDER_ITEMS → PAYMENTS → OUTBOX_EVENTS → AUDIT_LOG.
3. Implement ORDER_STATE_MACHINE procedure and AFTER UPDATE triggers.
4. Start MongoDB replica set (three nodes); enable authentication.
5. Create collections with all indexes: restaurants (compound), riderlocations (2dsphere + TTL), orderhistory (unique), eta_estimates (compound).
6. Install and test Outbox Sync Job.
7. Data of seed tests and run end-to-end integration tests.

Testing Plan

1. Unit Tests:

All state transitions of the PL/SQL tested individually - all six valid and at least four invalid paths.

2. Concurrency Test:

Repeat the previous steps with ten concurrent order placements to the same restaurant to confirm the Oracle row-level locking does not allow payment to be made twice.

3. Failure Recovery Test:

Kill a job that is already running in a batch; restart and verify that there are no duplicate or missing records in MongoDB orderhistory.

4. Performance Benchmark:

Compare the order history read latency of Oracle and MongoDB using the output of EXPLAIN / explain() and record the findings in the final report.

5. CAP Demonstration:

Isolate single MongoDB secondary; do reads with primaryPreferred versus secondaryPreferred; record difference in data to illustrate AP behaviour versus CP behaviour of Oracle [9].

References

[1] Elmasri, R. and Navathe, S. B. (2016) Fundamentals of Database Systems, 7th edn. Hoboken, NJ: Pearson Education.

[2] Dean, J. and Ghemawat, S. (2004) "MapReduce: Simplified Data Processing on Large Clusters," in Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI), San Francisco, CA, pp. 137–150.

[3] Sadalage, P. J. and Fowler, M. (2012) NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Upper Saddle River, NJ: Addison-Wesley.

[4] Kleppmann, M. (2017) Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA: O'Reilly Media.

[5] Fowler, M. (2011) "PolyglotPersistence," Martin Fowler's Blog [Online]. Available at: https://martinfowler.com/bliki/PolyglotPersistence.html (Accessed: April 2026).

[6] Chodorow, K. (2019) MongoDB: The Definitive Guide, 3rd edn. Sebastopol, CA: O'Reilly Media.

[7] Oracle Corporation (2021) Oracle Database Concepts 21c. Redwood Shores, CA: Oracle Technical Documentation.

[8] Richardson, C. (2018) Microservices Patterns: With Examples in Java. Shelter Island, NY: Manning Publications.

[9] Brewer, E. A. (2000) "Towards Robust Distributed Systems," in Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC), Portland, OR, p. 7.

[10] Feuerstein, S. (2014) Oracle PL/SQL Programming, 6th edn. Sebastopol, CA: O'Reilly Media.